



Assignment 1

Computernetze und ihre Leistung

von:

Michael Heise

Linus Henn

Luca Lars Stoeber

Matrikelnr.: 537385

Matrikelnr.: XXXXXX

Matrikelnr.: XXXXXX

Kurs:

Computernetze und ihre Leistung SoSe 2026

Dozent: **Ralph-Günther Holz**

Münster, 23. Mai 2026

Contents / Inhaltsverzeichnis

1	HTTP/1 und HTTP/2	1
1.1	Aufbau von TCP-Verbindungen und Durchführung von Anfragen	1
1.2	Analyse der HTTP/2-Kommunikation	1
1.3	Analyse der HTTP/1.1-Kommunikation	3
1.4	Vergleich der HTTP/1.1- und HTTP/2-Austausche	3
1.5	Erstellung von Pseudocode oder Ablaufbeschreibung eines HTTP/2-Clients über h2c	3
1.6	Werkzeuge und Methodik	3
1.7	Reflexion und Diskussion	3
2	SMTP	4

1 HTTP/1 und HTTP/2

1.1 Aufbau von TCP-Verbindungen und Durchführung von Anfragen

Für alle Anfragen wurde ein einfaches Python Script verwendet, welches die TCP-Verbindung aufbaut, die HTTP Anfrage sendet und die Antwort empfängt 1.

```
1 import asyncio
2 import httpx
3 import argparse
4
5 HTTP_1 = "1"
6 HTTP_2 = "2"
7
8 async def send_http_request(host, port, path, HTTP_version=HTTP_1):
9     http_1 = HTTP_version == HTTP_1
10    http_2 = HTTP_version == HTTP_2
11    async with httpx.AsyncClient(http1=http_1, http2=http_2) as client:
12        response = await client.get(f"http://{host}:{port}{path}")
13        print(f"{response.http_version} {response.status_code} {response.
14              reason_phrase}")
15        print("Headers:")
16        for header, value in response.headers.items():
17            print(f"    {header}: {value}")
18        print(f"Response Data: {response.text}")
19
20 if __name__ == "__main__":
21     parser = argparse.ArgumentParser()
22     parser.add_argument("--host", default="cuil.internet-transparency.org",
23                         help="Host to connect to")
24     parser.add_argument("--port", type=int, default=80, help="Port to
25                       connect to")
26     parser.add_argument("--path", default="/index.html", help="Path to
27                       request")
28     parser.add_argument("--http_version", choices=[HTTP_1, HTTP_2], default
29                         =HTTP_1, help="HTTP version to use (1 or 2)")
30     args = parser.parse_args()
31     asyncio.run(send_http_request(args.host, args.port, args.path, args.
32                                 http_version))
```

Code 1: Python Script zum Aufbau von TCP-Verbindungen und Durchführung von HTTP Anfragen

Das Script nutzt für die Anfragen die `httpx` Bibliothek [1], welche die TCP Verbindung für HTTP Anfragen vereinfacht. Es werden die HTTP Versionen 1.1 und 2 unterstützt, welche über die Kommandozeile ausgewählt werden können 2. Das Script gibt die Antwort des Servers auf der Kommandozeile aus.

1.2 Analyse der HTTP/2-Kommunikation

Die HTTP/2-Kommunikation zwischend dem Client und dem Server wurde über Wireshark analysiert. Dabei wurden die folgenden Schritte durchgeführt:

1. Starten von Wireshark und Auswahl der entsprechenden Netzwerkschnittstelle, um den Datenverkehr zu erfassen.
2. Anfrage an die Webseite mit dem Python Skript 1 senden, um die HTTP/2-Kommunikation zu initiieren.

```

1 usage: connection.py [-h] [--host HOST] [--port PORT] [--path PATH]
2                       [--http_version {1,2}]
3
4 options:
5   -h, --help            show this help message and exit
6   --host HOST           Host to connect to
7   --port PORT           Port to connect to
8   --path PATH           Path to request
9   --http_version {1,2} HTTP version to use (1 or 2)

```

Code 2: Ausgabe der Python script Optionen über den Befehl `python connection.py --help`

```
python3 connection.py --http-version 2
```

3. Aufgezeichnete Pakete nach der Anfrage durchsuchen.
4. Filterung der Frame Pakete Nummern um nur DNS Abfrage, TCP Handshake und HTTP/2 Pakete zu betrachten.


```
frame.number >= {ErstesPaketNummer} && frame.number <= {LetztesPaketNummer}
```
5. Speichern der relevanten Pakete für die weitere Analyse.
6. Analyse der HTTP/2 Pakete, um die Kommunikation zwischen Client und Server zu verstehen, einschließlich der Anfragen und Antworten, die über HTTP/2 gesendet wurden.

Über Wireshark können die versendeten Pakete im Detail nachverfolgt werden. Die Kommunikation beginnt dabei mit dem Aufbau einer TCP-Verbindung. Der Client fragt die TCP-Verbindung zum Server an, und der Server antwortet mit einem SYN-ACK-Paket. Der Client bestätigt den Verbindungsaufbau mit einem ACK-Paket, wodurch die TCP-Verbindung etabliert wird. Nach dem erfolgreichen Aufbau der TCP-Verbindung wird diese für die HTTP/2-Kommunikation genutzt, um Anfragen und Antworten zwischen Client und Server auszutauschen. Dabei sendet der Client ein Connection Preface, Settings und Window Update. Das Connection Preface ist ein einzigartiger String, der dem Server signalisiert, dass der Client HTTP/2 unterstützt, während die Settings und Window Update Pakete die Parameter für die Kommunikation festlegen. Die Settings enthalten Informationen über die unterstützten Funktionen und Einstellungen des Clients, während die Window Update Pakete die Flusskontrolle für die Datenübertragung regeln. In unserem Fall sind folgende Settings gesendet worden:

- Header table size : 4096
- Enable PUSH : 0
- Initial Windows size : 65535
- Max Frame size : 16384
- Max Concurrent Streams : 100
- Max Header List Size : 65536

Außerdem wurde folgendes Window Update Paket gesendet:

- Connection Window size (before) : 65535
- Connection Window size (after) : 16842751

- 1.3 Analyse der HTTP/1.1-Kommunikation
- 1.4 Vergleich der HTTP/1.1- und HTTP/2-Austausche
- 1.5 Erstellung von Pseudocode oder Ablaufbeschreibung eines HTTP/2-Clients über h2c
- 1.6 Werkzeuge und Methodik
- 1.7 Reflexion und Diskussion

2 SMTP

Literatur

- [1] Encode OSS. Http/2 support in httpx, 2026. URL: <https://www.python-httpx.org/http2/>.